

Compliance in Databases: A Study of Structural Policies and Query Optimization

Ahana Pradhan*, Srinivas Karthik**, Imtiyazuddin Shaik*, Srinivas Vivek*

*IIT Bangalore, India; **Microsoft, India

ahana.pradhan@iiitb.ac.in, srv@microsoft.com, shaik.imtiyazuddin@iiitb.ac.in, srinivas.vivek@iiitb.ac.in

Abstract

Growing privacy regulations and internal governance mandates are driving demand for fine-grained, context-sensitive access control in data management systems. Among competing approaches, content-based access control—where **access decisions depend on the data values referenced by a query**—is becoming particularly prominent, and is supported directly in modern database engines.

While simple content-based predicates often incur negligible overhead, increasingly rich policies can interact in subtle ways with query optimization, leading to significant and poorly understood performance variability. This paper investigates this gap by introducing a structural framework and expressive policy grammar for **modeling content-based compliance policies and analyzing their impact on query planning and execution in database systems**.

Building on this framework, we augment an analytical benchmark with structured policy workloads, enabling controlled evaluation of enforcement mechanisms and optimization strategies under combined query-policy workloads. Our experimental results show that policy structure has a decisive impact on optimizer behavior and end-to-end performance, underscoring the need for policy-aware database and optimizer design.

CCS Concepts

• Security and privacy → Information accountability and usage control.

ACM Reference Format:

Ahana Pradhan*, Srinivas Karthik**, Imtiyazuddin Shaik*, Srinivas Vivek*. 2026. Compliance in Databases: A Study of Structural Policies and Query Optimization. In *Workshop on Secure and Private Data Management (SeQureDB '26)*, May 31-June 05, 2026, Bengaluru, India. ACM, New York, NY, USA, 10 pages. <https://doi.org/10.1145/3807894.3810277>

1 Introduction

Data privacy has become a first-class concern in modern database systems. Regulations such as GDPR, HIPAA, and CCPA impose strict requirements on how organizations store, access, and process personal and sensitive data [6]. Beyond regulatory compliance, internal governance policies increasingly demand that access to data be conditioned not merely on a user's role or identity, but on the *content* of the data itself. For instance, a supplier record may be visible only if the supplier operates in an approved region; an order may be accessible only if it carries sufficient priority. This model

is known as **content-based access control** (aka *fine-grained* access control) [12, 13, 17]. **Here the access decisions are a function of the data values in a query, rather than a static privilege table.**

Content-based access control is increasingly supported as a native primitive in relational database engines. PostgreSQL implements it through Row-Level Security (RLS) [14], which allows administrators to attach per-table filter predicates that are automatically injected into every query accessing that table. Similar mechanisms exist in SQL Server [8], Oracle (VPD) [10], Snowflake [20], and other commercial systems. The appeal of native enforcement is clear: policies are defined once, applied transparently, and cannot be bypassed by application code.

The problem. Despite widespread adoption, the performance implications of content-based policies remain poorly understood. Simple policies—a single equality predicate or a range condition—are typically absorbed by the optimizer without measurable overhead. However, realistic governance scenarios require *compositional* policies: a policy on `lineitem` may depend on a policy on `partsupp`, which in turn depends on a policy on `supplier`. As such tiered, cross-table dependencies accumulate, the injected predicates grow in structural complexity. **Standard cost-based optimizers produce policy-violating plans in the majority of cases when compliance constraints are present** [3], underscoring that policy-unawareness is a fundamental optimizer design gap. When enforcement is added, the optimizer must simultaneously reason over the original query and injected policy subqueries—a task current planners were not designed for. The problem is further compounded by the diversity of enforcement mechanisms available: native RLS, security barrier views, query rewriting, and user-defined functions each impose different boundaries on what the optimizer can see and transform, making different trade-offs between security guarantees, optimization transparency, and implementation complexity.

The gap. The **absence of a structured framework** for generating and reasoning about policy complexity means that prior work either evaluates enforcement strategies on trivially simple policies that do not stress the optimizer, or on ad-hoc application-specific policies that resist generalization. There is no established benchmark or policy generation framework that allows controlled, reproducible evaluation of enforcement mechanisms as policy complexity scales.

Contributions: We address this gap through three contributions.

(1) **A structural policy generation framework.** At first glance, LLMs appear well-suited for policy generation given their NL2SQL capabilities. However, naive LLM-generated policies suffer from limited scope policies and cyclic/self-referencing policies, which are unusable through the basic enforcement methods.



This work is licensed under a Creative Commons Attribution 4.0 International License. *SeQureDB '26*, Bengaluru, India

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2219-6/2026/05

<https://doi.org/10.1145/3807894.3810277>

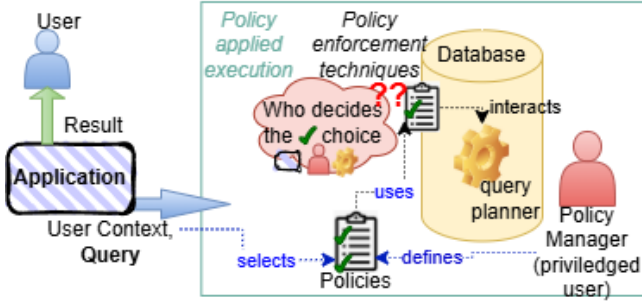


Figure 1: Interplay Between Users, Policies, and Databases

Motivated by this, we introduce a **structural model and expressive grammar for content-based compliance policies to capture a wide range of practical access control scenarios**. The framework explicitly supports both *white-box* enforcement—where policies are acyclic and admit cross query-policy optimization—and *black-box* enforcement—where cyclic, unrestricted policies are safely handled by treating policy predicates as opaque to the optimizer.

(2) **A policy-augmented benchmark.** We extend the TPC-H analytical benchmark with structured policy workloads generated using our framework. Policies are assigned business semantics over the TPC-H schema and systematically vary in composition depth, enabling controlled evaluation across a spectrum of complexity. The benchmark has been made publicly available¹.

(3) **A comparative empirical study.** We evaluate four enforcement strategies available in PostgreSQL—**Pure RLS (predicate injection & User Defined Functions)**, **Indexed RLS**, **Secure Views**, and **Query Rewrite via LITHE [5]**—under both acyclic and cyclic policy workloads, **measuring execution time, planning overhead, and robustness**. Our results show that policy structure primarily determines enforcement cost, that no single strategy dominates across all settings, and that policy-aware indexing and query rewriting offer the most reliable paths to scalable enforcement. *The evaluation lays the groundwork for a new class of policy-aware query optimizers that treat enforcement predicates as first-class optimization objects.*

2 A Framework for SQL Policy Formulation

2.1 Motivation

Query processing engines require a formal framework that treats regulatory policies as first-class citizens—essential for ensuring legal compliance, protecting privacy, enabling provable correctness, and providing scalable, uniform enforcement.

The framework’s complexity arises from the interplay of multiple stakeholders and competing concerns as captured in Figure 1. *Policy managers* define the compliance rules—often independently of the queries that will be subject to them—while *database administrators* must translate these rules into enforceable mechanisms within the engine. *End users* and application developers, in turn, expect transparent enforcement without changes to their query logic. This separation of concerns creates a rich design space: the same policy

can be enforced in multiple ways, each with different implications for security, performance, and implementation effort.

From a *security* standpoint, the choice of mechanism determines how much of the policy is visible to the optimizer. Black-box mechanisms—such as SECURITY DEFINER UDFs in PostgreSQL—fully hide policy logic from the planner, preventing information leakage through plan structure but forgoing cross query-policy optimization. White-box mechanisms—such as RLS and query rewriting—expose policy predicates to the optimizer, enabling better plans at the cost of increased attack surface for inference.

From a *performance* standpoint, policy persistence matters. Stable, long-lived policies justify the upfront cost of materialized views, policy-coverage indexes, or precompiled UDFs. Ad-hoc or frequently changing policies are better served by query rewriting, which incurs no materialization cost but demands a capable rewriter. Policy composition depth further amplifies this tension: tiered, multi-table policies generate deeply nested subqueries that stress cardinality estimation and join ordering regardless of the enforcement mechanism.

This interplay of stakeholder roles, security requirements, performance trade-offs, and enforcement diversity motivates the need for a principled policy framework. Current engines lack the abstractions to reason jointly about policy structure and query optimization—policies are either too shallow to reflect real governance requirements or too complex for the optimizer to handle efficiently. A formal framework spanning atomic to composite policies and white-box to black-box enforcement is therefore essential for understanding and closing this gap, which is described next.

2.2 Policy Model

Let $\mathcal{D} = (R_1, R_2, \dots, R_n)$ be a relational database schema, where each relation R_i has attributes $\text{attr}(R_i) = \{A_1, A_2, \dots, A_{k_i}\}$. Let I denote a database instance over $\mathcal{D}$. Let $\mathcal{U}$ denote the set of users (subjects) and C denote the execution context (including user identity, role, session attributes, and environment variables).

Semantics of “Policy” w.r.t. user query. In this paper, we restrict ourselves to policy logic expressible in SQL. Under this assumption, an *enforced-policy* can be viewed as a SELECT query: the WHERE clause (the σ operator) governs tuple visibility, while the SELECT clause (the π operator) governs attribute presentation, i.e., what the user can observe when the policy is evaluated. When multiple such policies are defined over a table, they can be combined through different logical compositions.

A policy $\alpha := \langle R, Q, \sigma, \pi, \mu \rangle$ represents a basic policy unit *on relation schema R* to restrict its data to the user who queries it. The semantics of $\langle R, Q, \sigma, \pi, \mu \rangle$ is a relational expression, which is explained using the following three definitions.

DEFINITION 1 (MASKING FUNCTION). A *masking function* $\mu : \text{dom}(A_j) \rightarrow \text{dom}(A_j) \cup \{\perp\}$ maps each value of attribute A_j of R either to a (possibly transformed) value in the same domain or to $\perp$, which denotes a suppressed value.

DEFINITION 2 (TUPLE VISIBILITY). A tuple $t \in R_i^I$ is visible to a user $u \in \mathcal{U}$ in context C if the enforcement mechanism does not suppress it from query results.

¹<https://github.com/imtiyazuddin/secure26db.git>

Therefore, a tuple $t \in R_i^I$ is visible under policy α if masking function μ does not map all attribute values of t to $\perp$.

DEFINITION 3 (ATTRIBUTE PRESENTATION). For a visible tuple t , an attribute A_j is presented faithfully if its original value in the database is returned to the user; it is transformed if a masking function $\mu(t.A_j)$ is applied (e.g., redaction, generalization, anonymization).

DEFINITION 4 (ATOMIC POLICY). Let R be a relation schema and let Q be a predicate over R . An atomic policy $\alpha = \langle R, Q, \sigma, \pi, \mu \rangle$ where:

- σ is the standard selection operator over R with predicate Q ,
- π is a standard projection operator over schema R ,
- μ is a masking projection operator over schema R , which, for each attribute $A_i \in R$, it applies a masking function f_i (Definition 1).

The semantics of α over a database instance D is defined as:

$$\llbracket \alpha \rrbracket_D = \pi(\sigma_Q(R^D)) \cup \mu(\sigma_{\neg Q}(R^D)).$$

Example set of atomic policies. Structurally, policy predicate query Q can differ based on its SQL constructs. In this paper, we specify the following kinds of atomic policy structures to illustrate the working principle of the framework.

- (1) **Attribute Predicate.** The class of queries constrain visibility using attribute value thresholds. Such policies arise naturally in geographic, organizational, and role-based compliance rules. One or more relation can be included in a policy, whose attribute values can be predicated.
- (2) **Existential Predicate.** Existence-based policies grant visibility if at least one related tuple satisfies a qualifying predicate. These policies correspond to *semi-join semantics*.
Note that attribute predicates – category (1) – can be expressed using EXISTS in SQL, as shown in the corresponding example in Appendix; they semantically enforce attribute constraints across joined relations, whereas existential predicates require only the existence of a qualifying related tuple.
- (3) **Universal Predicate.** Universal policy predicates enforce violation-free constraints: an entity is visible only if no related tuple violates the policy. These are typically expressed using NOT EXISTS and correspond to *anti-join semantics*.
- (4) **Grouping/Aggregated Predicate.** This class of policies enforces compliance based on statistics computed over groups of tuples, such as counts or distinct counts. These policies use GROUP BY and HAVING.
- (5) **Statistical Predicate.** Statistical predicated policies constrain higher-order statistics, often represented by nested aggregates, and other statistical functions beyond classical aggregates, capturing correlation-like relationships indicative of specific behavior.

Example SQLs for each of the above policy structures are listed in the Appendix on Page 9.

2.3 Policy Grammar: Composite Policies

To construct new policies by combining multiple atomic policies, a policy grammar is specified by the policy manager. For instance, we use the following grammar which allows logical combination of policies in an expressive way: A policy $\mathcal{P}$ is defined recursively by a context-free grammar. $\mathcal{P}$ is either a relational expression β or a conditional relational expression ι .

- A relational expression β may be: an atomic policy query α , a conjunction or a disjunction of the predicate queries of two atomic policies, or a negation of the predicate query of an atomic policy.
- A conditional expression ι has the form: **if Q then $\mathcal{P}_1$ else $\mathcal{P}_2$** , where the resulting policy depends on the truth value of the boolean predicate query Q .

$$\mathcal{P} ::= \beta \mid \iota$$

$$\beta ::= \alpha \mid (\beta \wedge \beta) \mid (\beta \vee \beta) \mid (\neg \beta)$$

$$\iota ::= \text{if } Q \text{ then } \mathcal{P} \text{ else } \mathcal{P}$$

$$\alpha ::= \langle R_i, Q, \sigma, \pi, \mu \rangle$$

To elaborate the composition using logical operators, let us take the instance of AND-ing two atomic policies $\alpha_1 = \langle R_1, Q_1, \sigma, \pi, \mu_1 \rangle$ and $\alpha_2 = \langle R_2, Q_2, \sigma, \pi, \mu_2 \rangle$. They are *compatible* to compose only when $R_1 = R_2$, i.e., both the policies are on the same relation. Consequently, π is the same identity projection for both α_1 and α_2 . The masking functions of μ_1 and μ_2 may be *different*. And lastly, predicates Q_1 and Q_2 are different to motivate a meaningful composition.

Therefore, the logical AND-ing has the following semantics:

$$\llbracket \alpha_1 \wedge \alpha_2 \rrbracket_D = \pi(\sigma_{Q_1 \wedge Q_2}(R^D)) \cup \mu_1(\sigma_{\neg Q_1}(R^D)) \cup \mu_2(\sigma_{\neg Q_2}(R^D)).$$

Similarly, the logical OR-ing makes a tuple fully visible when either of the policy predicates – Q_1 or Q_2 – holds. Only when both predicates fail does masking apply, and in that case both masking operators act on the region where the tuple satisfies neither predicate.

$$\llbracket \alpha_1 \vee \alpha_2 \rrbracket_D = \pi(\sigma_{Q_1 \vee Q_2}(R^D))$$

$$\cup \mu_1(\sigma_{\neg Q_1 \wedge \neg Q_2}(R^D)) \cup \mu_2(\sigma_{\neg Q_2 \wedge \neg Q_1}(R^D)).$$

Next, negation swaps the visibility and masking regions: tuples that satisfy Q are masked, while tuples that do not satisfy Q are made fully visible. Formally:

$$\llbracket \neg \alpha \rrbracket_D = \pi(\sigma_{\neg Q}(R^D)) \cup \mu(\sigma_Q(R^D)).$$

Note that the conditional expression can be equivalently written as implication using the other grammar rules. However, it adds CASE-WHEN-ELSE-THEN SQL-construct in this grammatical formulation, which eliminates the costly use of negation of a policy query. The above structural variations are sufficient to capture real-world analytic queries, as they capture flat SPJGAO, nested subqueries and correlations. Consequently, they are also sufficient to capture meaningful complex business rules.

Composite Policy Derivation Example. Let us elaborate on the policy derivation using an example.

Policy Intent. The visibility of a customer tuple depends on the customer's total spending and return behavior. A tuple is fully visible if the customer has spent more than 5000 in total and has no store returns. It is partially masked if the customer has spent more than 5000 but has at least one store return. In all other cases, the tuple is fully masked.

Base Relation. $R = \text{customer}$.

High-value predicate. $Q_1(c) :=$

```
(SELECT COALESCE(SUM(l.l_extendedprice * (1 - l.l_discount)), 0)
FROM orders o, lineitem l WHERE l.l_orderkey = o.o_orderkey
AND o.o_custkey = c.c_custkey) > 5000
```

Return predicate. $Q_2(c) :=$

```
EXISTS (SELECT 1 FROM orders o, lineitem l
WHERE l.l_orderkey = o.o_orderkey
AND o.o_custkey = c.c_custkey AND l.l_returnflag = 'R')
```

Atomic Policies. (μ_0 indicates no masking/transformation).

$$\alpha_1 = \langle R, Q_1 \wedge \neg Q_2, \sigma, \pi, \mu_0 \rangle$$

$$\alpha_2 = \langle R, Q_1 \wedge Q_2, \sigma, \pi, \mu_{\text{partial}} \rangle$$

$$\alpha_3 = \langle R, \neg Q_1, \sigma, \pi, \mu_{\text{NULL}} \rangle$$

Composed Policy $\mathcal{P} = \text{if } Q_1 \text{ then (if } Q_2 \text{ then } \alpha_2 \text{ else } \alpha_1) \text{ else } \alpha_3$
 $\llbracket \mathcal{P} \rrbracket_D = \pi(\sigma_{Q_1 \wedge \neg Q_2}(R^D)) \cup \mu_{\text{partial}}(\sigma_{Q_1 \wedge Q_2}(R^D)) \cup \mu_{\text{NULL}}(\sigma_{\neg Q_1}(R^D)).$

The corresponding SQL is listed in the Appendix.

Takeaway. The grammar captures a wide range of practical access control scenarios—from simple attribute filters to aggregate-guarded and statistically conditioned policies—while remaining grounded in standard relational algebra. Every policy admits a canonical AST representation that exposes its syntactic structure to program-transformation techniques, enabling predicate pushdown, redundancy elimination, and joint query-policy optimization.

3 Policy Enforcement Strategies

The compliance policies must be enforced inside the database engine to ensure correct behavior under any query. The following mechanisms are available in any standard database engine, which can be used to implement such policies.

- *RLS-based Enforcement.* Row-level security integrates policy checks directly into the database engine. Users write queries normally, and the system enforces the restriction transparently by adding the policy predicate to the query under the hood. Database engines such as PostgreSQL and SQL Server provide *row security policies* that automatically append policy predicates to query plans[14], [9]. Oracle provides the analogous mechanism through *Virtual Private Database (VPD)*, which dynamically appends security predicates based on session and application context [10].
- *Query Rewrite-based Enforcement.* While the policy predicate-injected user query is optimized natively by the database engine, a layer of semantic rewrite has the potential to improve the plan quality of the overall query. Consequently, a rewrite of the RLS-produced query is also a potential candidate mechanism.
- *View-based Enforcement.* Views enforce policies by embedding the compliance logic inside a predefined query. Users access the view instead of the base table, so the policy predicates are automatically applied. This keeps policy logic separate from user-written queries and makes the rules easy to see and audit at the schema level. Policies can also be composed by defining views on top of other views without changing the underlying tables.
- *User-Defined Function-based enforcement.* UDFs can be used to modularize policy logic by allowing it to be defined once and invoked uniformly across user queries. This promotes reuse and simplifies maintenance: when a regulation changes, only the UDF implementation needs to be updated. In addition, UDFs can express fine-grained restrictions that depend on runtime identity, session state, or other contextual information, thereby enabling dynamic and context-aware enforcement.

3.1 White-box and Black-box Enforcement

Enforcement methods fall into two broad categories. In *white-box* enforcement, the policy predicate is visible to the query optimizer, enabling joint query-policy optimization; predicate pushdown, index exploitation, and partition pruning are all applicable. RLS, query rewriting, views, and transparent UDFs belong to this class. In *black-box* enforcement, the policy logic is opaque to the planner—typically for security reasons (side channel or error-based inference attacks) where users lack permission to inspect the policy—disabling cross-policy optimization. This class includes opaque views and engine-specific black-box UDFs.

3.2 Note on Policy Structure

Cyclic. A policy predicate may reference the very table to which it is attached (*direct self-reference*), or it may induce a reference cycle through a chain of intermediate tables (*multi-hop self-reference*, e.g. $t_1 \rightarrow t_2 \rightarrow \dots \rightarrow t_1$). In either case, the enforcement mechanism re-invokes the policy predicate while evaluating it, typically manifesting as an *infinite recursion* error during query planning [11]. Database engines differ in their ability to detect such cycles prior to execution: most of the planners perform static cycle detection and raise an error at policy-compilation time.

Acyclic: Tiered Policy Generation. It is imperative that the policy generation also allow a systematic way to prevent cyclic policy structure. We have the following rule to achieve that.

Let G_s be the schema dependency DAG, $A(t)$ be the set of ancestor tables of table t in G_s , and P_t be the policy selected for table t . Let B denote the set of base tables. Policy generation is constrained by $\text{Refs}(P_t) \subseteq A(t) \cup B$.

The tier assignment is then defined recursively as

$$\text{tier}(t) = \begin{cases} 0 & \text{if } t \in B, \\ 1 + \max_{u \in \text{Refs}(P_t)} \text{tier}(u) & \text{otherwise.} \end{cases}$$

Because every policy reference points only to tables in strictly lower tiers, the resulting policy-dependency graph is acyclic.

4 Experimental Analysis

Objective. We instantiate our policy framework to generate practically meaningful, compositionally structured policies over the TPC-H benchmark, enforced through the standard analytical query workload using the implementation strategies discussed in Section 3. While the enforcement mechanisms are individually well-understood, its behavior under complex, nested policy compositions remains unstudied. Our goal is to close this gap by systematically characterizing the optimization behavior and performance limits of each strategy as policy complexity increases. Our code has been made publicly available².

System Setup: Our setup includes $2 \times$ Intel(R) Xeon(R) Silver 4114 CPUs @ 2.20GHz, 128 GB DDR4 RAM, and 8 TB HDD, running Ubuntu 24.04.4 LTS. We experimented with opensource **PostgreSQL 18.2** and a commercial engine: Com-eng A³. We use the standard TPC-H benchmark at scale factor 1 for the underlying

²<https://github.com/imtiyazuddin/secure26db.git>

³Name anonymized due to the "Dewitt" Clause

database.⁴ PostgreSQL requires explicit tuning, which we perform using the standard PG Tune configuration. By default, experiments use standard primary-key and foreign-key indexes, given the OLAP focus of the user query workload.

4.1 Policy Generation

We generate content-based policies over the TPC-H schema using our framework’s policy model, grammar, and tiered acyclic composition structure. Policies are designed to carry genuine business semantics and realistic governance scenarios. We leverage the NL2SQL capabilities of GPT-5 by encoding the policy model, grammar rules, and tiered structure as structured prompts, guiding the model to produce syntactically valid and semantically meaningful SQL policy predicates. Each generated policy is validated against the TPC-H schema for correctness and the above mentioned constraints into the evaluation workload. We generate a suite of policies, the full list of which is available in the Git repository. A representative subset used in our evaluation is described below.⁵

4.1.1 Acyclic Policies. Due to some engine limitation for handling cyclic policies, or ending up in infinite recursion, we generate the following non-cyclic policies:

- **Customer:** Only customers who have high purchasing power, and only if they belong to the approved business regions.
- **Supplier:** Only suppliers with a good financial standing are included, and they must belong to the Europe region.
- **Partsupp:** Only supplier-part inventory records are included when the available stock is plentiful, and the linked supplier is financially stable.
- **Lineitem:** Only transaction line items are included when they are associated with high-priority orders and with part-supplier combinations that have plenty of stock.

4.1.2 Cyclic Policies. We also consider *unrestricted* policies that permit cyclic inter-policy dependencies, which arise naturally in certain applications such as mutual visibility constraints or bidirectional access rules. While such cycles preclude the tiered acyclic structure assumed earlier, PostgreSQL’s SECURITY DEFINER feature provides a safe enforcement path: breaking the recursive policy application chain and preventing infinite evaluation loops.

- **Orders:** Only high-value orders are included. In business terms, an order is visible only if its total discounted sales value exceeds a defined revenue threshold.
- **Customer:** Only customers who have participated in cross-country business within the same region are included.
- **Part:** Only parts that have not been associated with unusually large individual order quantities are included.
- **Supplier:** Only suppliers are included who are not linked to low-priced products below a defined price threshold.

4.2 Policy Enforcement Methods: PostgreSQL

As discussed in Section 3, we evaluate the following policy-enforcement strategies available in PostgreSQL:

⁴We also conducted experiments with scale factor 10, but primarily report results at scale factor 1 due to the high runtime caused by query complexity induced via policies.

⁵In this work, we limit one policy per base table due to limitation in PostgreSQL and other engines

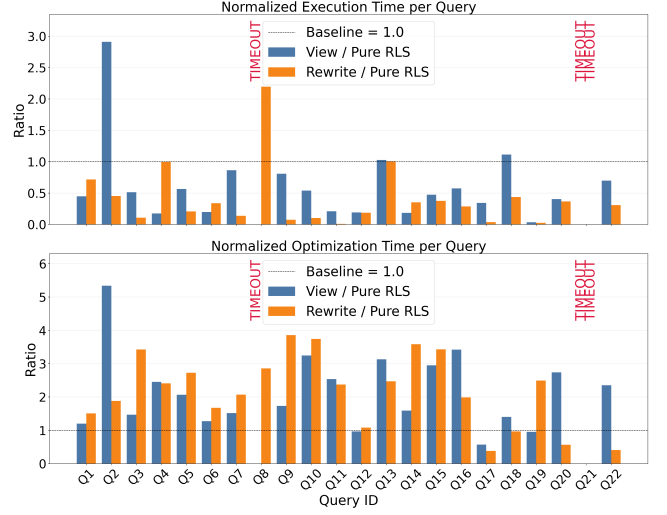


Figure 2: RLS vs. View vs. Query Rewrite for Acyclic Policies

- (1) **Pure RLS / Indexed RLS:** Policies are enforced as Row-Level Security predicates, which PostgreSQL injects directly into query execution at the base-table level. To assess the benefit of planner-side optimization, we additionally evaluate **Indexed RLS**, where *policy-coverage indexes* are created to accelerate evaluation.
- (2) **Secure Views:** Since policies are typically stable and well-defined, each policy query is a view, and the original user query is rewritten to reference these views in place of the base tables.
- (3) **Query Rewrite:** Policy predicates are inlined directly into the user query, and the resulting combined query is rewritten using the state-of-the-art rewriter LITHE [5], enabling cross-predicate optimization across the policy and query logic.
- (4) **UDF:** PostgreSQL User-Defined Functions (UDF) defined with SECURITY DEFINER treat the policy as an opaque black box, disabling cross-optimization between the policy and the user query. While transparent UDFs permit some cross-optimization, their performance is consistently inferior to query rewrite; we therefore include only the security definer variant in the evaluation.

Note that each of the above methods when referred to in the rest of the section is, by default, evaluated in a **white-box setting**, which enables cross query-policy optimization. When this is not allowed, we denote it by **black-box setting**. Our experiments do not use masking policies as they cannot be enforced using RLS.

4.3 Experiments & Observations: PostgreSQL

4.3.1 Cross Query-Policy Optimization: Enabled. We first evaluate enforcement strategies that treat policies as white-box, enabling cross-optimization between policy predicates and the user query. This includes Pure RLS, Views, and Query Rewrite via LITHE. Since policies are structurally exposed to the optimizer, cyclic policies are excluded from this setting to avoid infinite recursion during predicate injection. Figure 2 reports normalized execution time (top) and optimization time (bottom) for Views and Query Rewrite, both normalized to Pure RLS as the baseline (ratio < 1 indicates faster than Pure RLS).

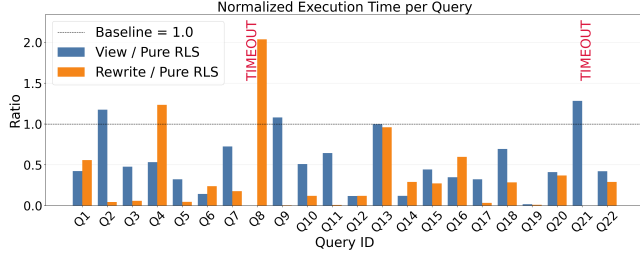


Figure 3: RLS vs. View vs. Rewrite on scale factor 10 data

Execution time. Query Rewrite consistently outperforms Pure RLS across the majority of queries, confirming that explicit predicate inlining followed by algebraic simplification via LITHE allows the optimizer to make better join-ordering and filter-pushdown decisions than the implicit predicate injection used by RLS. The gains are most pronounced on join-heavy queries with correlated policy predicates—Q3, Q7, Q9, Q10, Q11, Q17, and Q19—where Pure RLS fails to exploit cross-predicate optimization opportunities. However, Query Rewrite times out on Q8 and Q21, indicating that aggressive inlining of deeply nested, tiered policies can produce query trees too large for the planner to handle within practical limits. In terms of overhead, LITHE uses a small number of LLM iterations to converge on the best rewritten query, which is a one-time cost per query-policy combination.

Overall, Views also achieve significant execution improvements over Pure RLS on several queries—Q4, Q6, Q11, Q12, Q14, and Q19. The view cost is a one-time effort that is practically acceptable given that policies change infrequently. The fact that Query Rewrite outperforms Views on several queries reveals missed optimization opportunities by the planner.

Optimization time. Pure RLS spends the least planning time across most queries, as the injected predicates remain compact and do not expand the planner’s search space. Both Views and Query Rewrite incur substantially higher planning overhead—often 2–4× that of Pure RLS—since view introduces additional scan nodes while query rewriting produces larger, more complex query trees that broaden the planner’s enumeration space. This is the central tension of the white-box approach: better execution comes at the cost of higher planning overhead.

Effect of indexing. Indexed RLS yields only marginal improvement over Pure RLS. The acyclic policy predicates used here do not exhibit selectivities favorable for index use; moreover, acyclic policies are structurally simpler and largely lack the complexity that could benefit from index-based access paths.

Impact of Scale. To assess sensitivity to data volume, we repeat the evaluation at TPC-H scale factor 10 and report results in Figure 3. The relative ordering of the three strategies is preserved: both Views and Query Rewrite continue to outperform Pure RLS in execution time and robustness. Notably, the per-query performance profile remains largely stable across scale factors, suggesting that the observed behavior is driven by policy structure rather than data volume. There are, however, a few scale-dependent surprises: Q21 completes without timeout under Views at SF-10 (unlike at

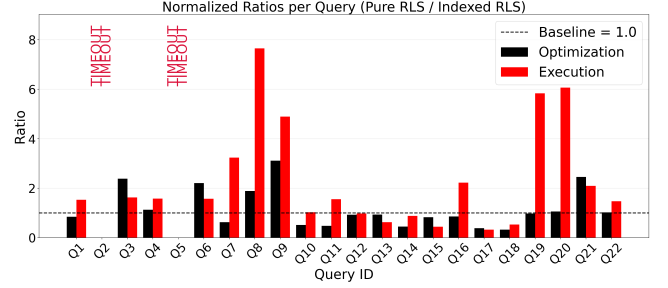


Figure 4: Impact of Policy-Aware Indexing on Black-box RLS

SF-1), and the View slowdown on Q2 narrows from approximately 3× to under 1.5×. These exceptions suggest that at larger scales, increased cardinality may influence the optimizer plan better.

Takeaway. Query Rewrite via LITHE is the most effective enforcement strategy under the white-box setting, delivering consistently lower execution times at the cost of higher planning overhead and occasional planner timeouts on the most complex queries. Views offer a practical middle ground—stable and easier to deploy but leave missed optimization opportunities on the table. One can also think of using the above two techniques in conjunction for more performance benefits. Pure RLS, while the most predictable and planner-friendly, pays a systematic execution penalty as policy complexity grows. Finally, the relative performance profile of each strategy is largely independent of data volume, with only isolated queries showing scale-dependent behavior – reinforcing that policy structure is the primary determinant of enforcement overhead.

4.3.2 Cross Query-Policy Optimization: Disabled. We now turn to the setting where cross-optimization between queries and policies is disabled. As discussed in Section 3, this is the appropriate choice when policies must be treated as black-box for security reasons. The opacity of the policy boundary prevents the engine from recursively expanding policy predicates, which also means PostgreSQL does not enter infinite recursion on cyclic policies. This setting therefore naturally accommodates unrestricted, cyclic policy structures, which are evaluated here. Note that in this scenario, UDF and pure RLS (black-box version) become synonymous.

Effect of indexing. In the acyclic setting, indexing offered negligible benefit. We now investigate whether cyclic policies—which are structurally more complex—create conditions where policy-aware indexing can provide meaningful speedup. In the Indexed RLS (black-box) configuration, indexes are created on all individual columns referenced by the policy queries. Figure 4 reports normalized execution and optimization time ratios of Pure RLS (black-box) with respect to Indexed RLS (black-box).

The most important result is the dominant overhead of Pure RLS (black-box) in execution. This pattern is most pronounced on queries that activate multiple policies simultaneously. Q8, which triggers all four policies at once, exhibits one of the largest execution slowdowns in the workload. Similarly, Q5, Q7, and Q9, each combining three policies, are considerably more expensive under Pure RLS (black-box). Query Q20 combines two policies — also show elevated normalized execution times. Lastly, although Q19

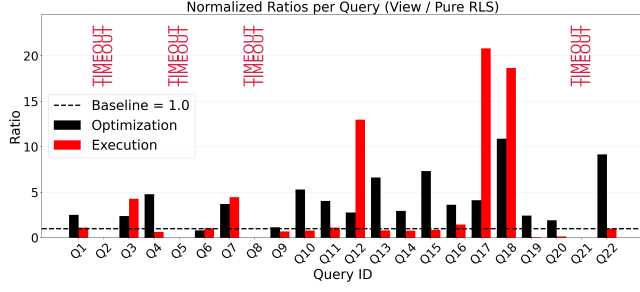


Figure 5: Black-box RLS vs. View for Cyclic Policy

has only one policy, it uses a universal policy predicate, which evaluates a correlated predicate and then negates it, leading to high runtime. This policy also contributes to the slowdown of Q20. Taken together, these results establish that policy composition is a primary driver of execution overhead and that indexing provides meaningful relief as compositional complexity grows.

Importantly, **Pure RLS (black-box) times out on Q2 and Q5, while Indexed RLS (black-box) completes both successfully.** This demonstrates that the benefit of policy-aware indexing extends beyond average latency reduction—it also improves robustness and completion reliability on the most demanding queries. That said, Indexed RLS (black-box) does not uniformly dominate: Q13, Q15, Q17, and Q18 all execute faster under Pure RLS (black-box), reflecting query-specific plan choices where additional indexes do not align with the preferred access patterns. Nevertheless, across the full workload, Indexed RLS (black-box) offers a markedly better worst-case profile and more stable completion behavior.

Optimization-time behavior is more mixed: the additional indexes do not consistently increase or decrease planning overhead, suggesting that index availability alone does not systematically perturb the planner’s search process.

Black-box Pure RLS vs. Views: Unrestricted Policies. Figure 5 compares Pure RLS (black-box) and Security Barrier Views across planning and execution time for the unrestricted (cyclic) policy set. To handle cycles safely, RLS predicates are wrapped in SECURITY DEFINER functions [16], and views as security barrier views [15], which impose the same kind of optimization boundary. Since LITHE does not respect such boundaries, query rewrite is excluded from this experiment. Pure RLS (black-box) provides more stable trade-off across the workload, while views introduce consistently higher planning overhead and more variable execution behavior.

$p(o)$	$\neg p(o)$
$(o_orderstatus = 'F') \wedge$ $(o_totalprice > 100000) \wedge$ $(o_orderdate \geq 1995-01-01)$	$(o_orderstatus \neq 'F') \wedge$ $(o_totalprice \leq 100000) \wedge$ $(o_orderdate < 1995-01-01)$
Mechanism	Access path
Predicate Injection	Parallel semi-joins
Security Definer Policy	Index-only scan + UDF filter
	Slowdown
	1.00×
	55.09×

Table 1: Case with $p(o) \wedge \neg p(o)$ Policy

Takeaway. Under black-box setting, policy-aware indexing significantly improves both robustness and worst-case execution latency for cyclic policies, while Pure RLS (black-box) remains more stable and planning-efficient than Security Barrier Views. Policy composition depth is the dominant cost driver in this setting. Future optimizers should use cost-based optimization to select between view-based and pure RLS approaches for a given query.

$p(o)$	$q(o)$
$(o_totalprice > 150000) \wedge$ $(o_orderstatus = 'F') \wedge$ $(o_orderdate > 1994-12-31) \wedge$ $\exists o_2 (o_2.o_custkey = o.o_custkey$ $\wedge o_2.o_orderstatus = 'F')$	$(o_totalprice > 50000) \wedge$ $(o_orderdate > 1993-12-31) \wedge$ $\exists o_2 (o_2.o_custkey = o.o_custkey$ $\wedge o_2.o_orderstatus = 'F')$

Table 2: Atomic Policies for Composition

Predicate Composition	Security Definer	Predicate Injection
$p \wedge \neg q$	1.00×	1.00×
$\neg q \wedge p$	2.74×	1.83×

Table 3: Slowdown caused by predicate order

Order	Mechanism	Access path	Slowdown
$p \wedge \neg q$	Security Definer Policy	Parallel index-only scan using $o_orderstatus$ index, partial/final aggregate	1.00×
$p \wedge \neg q$	Predicate Injection	Bitmap heap scan with $Bi\ tmapAnd$ over all indexes, + one subplan	8.62×
$\neg q \wedge p$	Security Definer Policy	Parallel index-only scan using $o_orderstatus$ index, partial/final aggregate	1.00×
$\neg q \wedge p$	Predicate Injection	Bitmap heap scan with $Bi\ tmapAnd$ over all indexes, + two subplans	5.75×

Table 4: Impact of Policy-Coverage Indexing

4.3.3 Stress-testing Optimizers with Policy Complexity. In this category of experiments, we test how the RLS mechanism handles the simplification of logical connections among policy predicates. We take the fastest implementation as 1x and compare the relative slowdown of the other. First, we check the logical AND-ing of contradictory policies. The predicates are on orders tables, as in Table 1. We have a composite policy $p(o) \wedge \neg p(o)$, where each policy is either implemented using (i) security definer function to be used as RLS, or (ii) manual predicates injected with the user query – equivalent to standard RLS. Both versions use only PK+FK indexes. The main observation is listed in Table 1. RLS enabled with black-box functions hides predicate semantics inside scalar functions. As a result, PostgreSQL cannot rewrite the logic into joins; it performs an index-only scan over all 1.5M rows and applies the opaque function filter tuple-by-tuple. This yields the worst behavior despite zero heap fetches. On the other hand, when the predicates are visible in the query plan, the planner converts the same test into set-oriented parallel semi-joins, so the engine evaluates p and q once as relations and combines them efficiently. Therefore, predicate visibility and set-based rewriting matter far more than nominal index use.

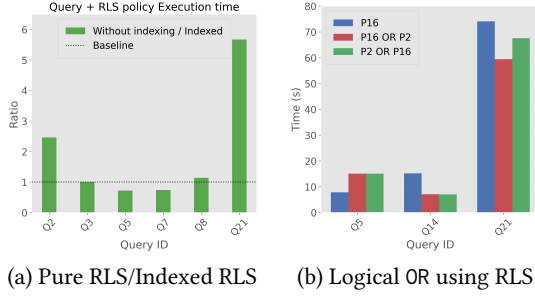


Figure 6: RLS Policy performance in Comm-Engine A

Next, we study a composite policy that involves negations. The component predicates are listed above. In the composition, we test $\text{EXISTS } p(o) \wedge \text{NOT EXISTS } q(o)$ where both orders of the predicates are tested for each mechanism. We also used *policy-coverage indexes here*. The results are listed in Table 3. The key observation is that order of the NOT predicate causes severe execution slowdown, in both the mechanisms. In predicate injection, there was no change in the plan shape. It is consistent with executor-level variance (e.g., Gather/parallel coordination) rather than a true optimization effect. Because q is subsumed by p , $\neg q$ was not executed, hence, obtaining faster result. On the other hand, black box functions could not have such optimization, thereby gaining higher slowdown.

One more interesting point here is the effect of indexing. Predicate injection is substantially slower than the security-definer formulation for both predicate orders, as listed in Table 4. The security-definer/RLS plan uses a parallel index-only scan. In contrast, injected predicates used a bitmap heap scan and large subplans.

4.4 Commercial Database Engine

We also looked at a Com-Engine A, specifically on the impact of index and order of predicates in logical combination in a feature similar to PostgreSQL RLS. Similar to PostgreSQL, the injected predicates also effectively use the indexes, as can be observed in Figure 6(a). Moreover, we investigate the effect of predicate ordering in the logical OR combination, using one simple and one expensive predicate – labeled as P2 and P16, respectively, in Figure 6(b). It is worth noting that the combined policy incurs both faster and slower execution times, depending on the query. Also, the order of the predicate matters in most of the cases.

5 Related Work

Prior work on database compliance spans policy-aware query processing, query-time compliance checking, disclosure control, and policy specification. Under our framework, GDPRbench [18], and XACML [2] primarily capture *attribute-predicate* policies from our atomic policy structures: GDPRbench models GDPR rights such as consent, erasure, portability, and purpose limitation as SQL workloads; XACML provides a general declarative ABAC language. Beedkar et al. [3] and Schwab et al. [19] use both *attribute* and *grouping/aggregate* predicates: the former introduce compliant geo-distributed query processing, in which SQL-dataflow policies are incorporated into query planning as location constraints, while the

latter checks query compliance against policy rules using similarity-based matching. Mascara [13] enforces data disclosure policies by rewriting SPJG SQL queries and applying intuitive *data masks* for context-aware partial disclosure of sensitive attributes based on user roles. The *transformation enforcement action* – μ functions that do to map to $\perp$ – captures the same semantics of data-masking. Finally, AND/OR composition of predicates is also observed in Sieve [12], which scales fine-grained enforcement for large GDPR/CPA rule sets through query-context filtering. However, Sieve derives its policy rules for IoT use cases, where the rules are predominantly simple value-based predicates. Composition of security policies is approached in the security domain, such as in SePL [7]. The composition framework is a modal logic-based language that allows union (least restrictive), intersection (most restrictive), prioritized override, sequential chaining, and conditional activation of diverse actions (read/write, execute, connect, delegate, encrypt/decrypt)s. Our framework brings a similar capability of combining policy queries in the database context. Similar contrast can be found in other security policy frameworks, such as in [1] and [4].

6 Conclusions & Future Work

We presented a framework for generating and reasoning about structurally complex content-based compliance policies, through a formal policy model and expressive grammar that supports both *white-box* and *black-box* enforcement methods. Using this framework, we augmented TPC-H with structured policy workloads and evaluated four PostgreSQL enforcement strategies. Our central finding is that **policy structure is the primary driver of enforcement overhead, and that no single strategy dominates across all settings**. These results expose a fundamental gap in current optimizer design. Based on our findings, we identify several directions for future work and database engine design. On the system side, actionable improvements include: (i) **policy-aware cardinality estimation, where the planner models policy predicate selectivity jointly with query predicates**; (ii) a policy-aware index advisor that recommends indexes on policy-referenced columns as a first-class optimization target; (iii) partial predicate pushdown through security barriers, relaxing the current all-or-nothing barrier semantics for well-typed, non-sensitive predicates; and (iv) enabling the engine to natively select black-box enforcement for cyclic RLS-policies without requiring manual definitions of black-box wrappers.

On the formalism side, our current model assumes atomic policies act on disjoint tuple sets; relaxing this to support overlapping scopes will require priority or conflict-resolution rules. Finally, query-rewriting-aligned techniques that translate compliance policies into regulation-aware transformations while preserving execution robustness remain an important open problem.

Acknowledgment

This work was partially funded by the Privacy-Preserving Data Processing and Exchange for Sensitive Data in the National Digital Public Infrastructure (P3DX) project of Srinivas Vivek.

Declaration of using AI

We used popular GPT models for coding and writing assistance but assume full responsibility for the accuracy of the work.

References

- [1] Enterprise privacy authorization language. Wikipedia. Formal language for enterprise privacy policies.
- [2] extensible access control markup language (xacml). OASIS Standard, 2013. Declarative attribute-based access control policy language and architecture.
- [3] K. Beedkar, D. Brekardin, J.-A. Quiané-Ruiz, and V. Markl. Compliant geo-distributed query processing in action. In *Proceedings of the VLDB Endowment*, volume 14, pages 2843–2846. VLDB Endowment, 2021.
- [4] N. Damianou, N. Dulay, E. Lupu, and M. Sloman. Ponder: A language for specifying security and management policies for distributed systems. Technical report, Imperial College Research Report, 2001.
- [5] S. Dharwada, H. Devrani, J. R. Haritsa, and H. Doraiswamy. LITHE: A query rewrite advisor using llms. In *Proceedings 29th International Conference on Extending Database Technology, EDBT 2026, Tampere, Finland, March 24-27, 2026*, pages 233–246. OpenProceedings.org, 2026.
- [6] C. J. Hoofnagle, B. van der Sloot, and F. Zuiderveen Borgesius. The european union general data protection regulation: What it is and what it means. *Information & Communications Technology Law*, 28(1):65–98, 2019.
- [7] M. Mejri and H. Yahyaoui. Formal specification and integration of distributed security policies. *Computer Languages, Systems & Structures*, 49:1–35, 2017.
- [8] Microsoft. Row-level security (sql server). <https://learn.microsoft.com/en-us/sql/relational-databases/security/row-level-security>, 2024. Accessed: 2026-02-22.
- [9] Microsoft Corporation. Row-level security, 2025. Microsoft Learn documentation.
- [10] Oracle. Using Oracle Virtual Private Database to Control Data Access, 2025. Official Oracle documentation on Virtual Private Database (VPD): policies attach predicates to tables, enforcing row/column access control.
- [11] Oracle Corporation. *DBMS_RLS*. Oracle Corporation, 2019. Accessed: 2026-03-11.
- [12] P. Pappachan, R. Yus, A. Fard, and S. Mehrotra. Sieve: A middleware approach to scalable access control for database management systems. In *Proceedings of the VLDB Endowment*, volume 13, pages 2424–2437. VLDB Endowment, 2020.
- [13] R. Poepsel-Lemaitre, K. Beedkar, and V. Markl. Disclosure-compliant query answering. *Proceedings of the ACM on Management of Data*, 2(6):233, 2024.
- [14] PostgreSQL Global Development Group. *Documentation: Row Security Policies*, 2025. Official documentation describing PostgreSQL RLS implementation and semantics.
- [15] PostgreSQL Global Development Group. *PostgreSQL 18 Documentation: CREATE VIEW*, 2026. Accessed: 2026-03-11.
- [16] PostgreSQL Global Development Group. PostgreSQL Documentation: CREATE FUNCTION. <https://www.postgresql.org/docs/current/sql-createfunction.html>, 2026. See the SECURITY DEFINER / SECURITY INVOKER function attribute.
- [17] S. J. Rizvi, A. O. Mendelzon, S. Sudarshan, and P. Roy. Extending query rewriting techniques for fine-grained access control. In *Proceedings of the ACM SIGMOD International Conference on Management of Data*, pages 551–562. ACM, 2004.
- [18] I. Saeed, Q. Mumtaz, P. Boncz, and S. Al-Kiswani. Understanding and benchmarking the impact of gdpr on database workloads. *IEEE Data Engineering Bulletin*, 42(4):3–15, 2019.
- [19] P. K. Schwab, J. Röckl, M. S. Langohr, and K. Meyer-Wegener. Performance evaluation of policy-based sql query classification for data-privacy compliance. *Datenbank-Spektrum*, 21(3):191–201, 2021.
- [20] Snowflake. Understanding row access policies. <https://docs.snowflake.com/en/user-guide/security-row-intro>, 2025.

Appendix

Examples of Atomic Policy Predicates wrt Section 2.2

(1) **Attribute Predicate:** In TPC-H setting, a customer may be allowed to see only those lineitems that belong to her orders and are supplied by suppliers from the same nation as the customer. Therefore, we have $R = \text{lineitem}$, and π is the identity projection over all attributes of lineitem . Using l as its alias, the policy query corresponding to the authorized tuples of lineitem is given as follows:

```
EXISTS (SELECT 1
FROM orders o, customer c, supplier s
WHERE l.l_orderkey = o.o_orderkey
AND o.o_custkey = c.c_custkey
AND l.l_suppkey = s.s_suppkey
AND c.c_name = :current_user
AND c.c_nationkey = s.s_nationkey)
```

Existential Predicate: Any query may be allowed to fetch data of only those customers who have placed at least one high-value order. Therefore, we have

$R = \text{customer}$, and π is the identity projection over all attributes of customer . Using c as its alias, $Q(c)$ is given as follows:

```
EXISTS (SELECT 1 FROM orders o
WHERE o.o_custkey = c.c_custkey AND o.o_totalprice > 1000)
```

Universal Predicate: Any query may be allowed to fetch data of only those customers who have not returned any ordered item. Therefore, we have $R = \text{customer}$, and π is the identity projection over all attributes of customer . Using c as its alias, $Q(c)$ is given as follows:

```
NOT EXISTS (SELECT 1
FROM orders o, lineitem l
WHERE o.o_custkey = c.c_custkey
AND l.l_orderkey = o.o_orderkey
AND l.l_returnflag = 'R')
```

Grouping/Aggregate Predicate: Any query may be allowed to fetch data of only those customers who have purchased items from at least three distinct suppliers. Therefore, we have $R = \text{customer}$, and π is the identity projection over all attributes of customer . Using c as its alias, $Q(c)$ is given as follows:

```
EXISTS (SELECT 1
FROM orders o, lineitem l
WHERE o.o_custkey = c.c_custkey
AND l.l_orderkey = o.o_orderkey
GROUP BY o.o_custkey
HAVING COUNT(DISTINCT l.l_suppkey) >= 3)
```

Statistical Predicate: Queries may be allowed to use data of only those lineitem tuples whose discounted revenue contributes at least 20% of the total revenue of their corresponding order. Therefore, we have $R = \text{lineitem}$, and π is the identity projection over all attributes of lineitem . Using l as its alias, $Q(l)$ is given as follows:

```
l.l_extendedprice * (1 - l.l_discount) >= 0.2 * (
SELECT SUM(l2.l_extendedprice * (1 - l2.l_discount))
FROM lineitem l2
WHERE l2.l_orderkey = l.l_orderkey)
```

SQL for the Composite Policy in Section 2.3

Here, customer_status is the relation obtained by evaluating $\mathcal{P}$. CTEs customer_spending captures the evaluation of the LHS of the policy predicate Q_1 , and customer_returns captures Q_2 . The three disjoint selections $\sigma_{Q_1 \wedge \neg Q_2}(R^D)$, $\sigma_{Q_1 \wedge Q_2}(R^D)$, and $\sigma_{\neg Q_1}(R^D)$ are present inside the body of this CTE. Finally, the projections π , μ_{partial} , and μ_{NULL} correspond to the SELECT-clause applied on customer_status , in the final part of the query.

```
WITH customer_spending AS (
SELECT
c.*,
COALESCE(SUM(l.l_extendedprice * (1 - l.l_discount)), 0)
AS total_spent
FROM
customer c, orders o, lineitem l
WHERE
c.c_custkey = o.o_custkey
AND o.o_orderkey = l.l_orderkey
GROUP BY
c.c_custkey, c.c_name, c.c_address,
c.c_nationkey, c.c_phone, c.c_acctbal,
c.c_mktsegment, c.c_comment
),
customer_returns AS (
SELECT DISTINCT
o.o_custkey
FROM
orders o,
lineitem l
WHERE
o.o_orderkey = l.l_orderkey
AND l.l_returnflag = 'R'
),
customer_status AS (
SELECT
cs.*,
CASE
WHEN cs.total_spent > 5000
AND cr.o_custkey IS NULL
```

```

        THEN 'FULL'
      WHEN cs.total_spent > 5000
        AND cr.o_custkey IS NOT NULL
      THEN 'PARTIAL'
    ELSE 'NONE'
  END AS visibility_level
FROM
  customer_spending cs
  LEFT JOIN customer_returns cr
    ON cs.c_custkey = cr.o_custkey
)
SELECT
  c_custkey,
  c_name,

  CASE
    WHEN visibility_level = 'FULL'
    THEN c_address
    WHEN visibility_level = 'PARTIAL'
    THEN 'MASKED'
    ELSE NULL
  END AS c_address,

  CASE
    WHEN visibility_level = 'FULL'
    THEN c_phone
    WHEN visibility_level = 'PARTIAL'

```

```

        THEN 'MASKED'
    ELSE NULL
  END AS c_phone,

  CASE
    WHEN visibility_level IN ('FULL', 'PARTIAL')
    THEN c_nationkey
    ELSE NULL
  END AS c_nationkey,

  CASE
    WHEN visibility_level IN ('FULL', 'PARTIAL')
    THEN c_acctbal
    ELSE NULL
  END AS c_acctbal,

  CASE
    WHEN visibility_level IN ('FULL', 'PARTIAL')
    THEN c_mktsegment
    ELSE NULL
  END AS c_mktsegment,

  CASE
    WHEN visibility_level IN ('FULL', 'PARTIAL')
    THEN c_comment
    ELSE NULL
  END AS c_comment
FROM customer_status;

```